

Розробка 2D-ігри на C++ та SFML

від Процедурного коду до ООП

Як додавати нові ігрові механіки та сутності, не ламаючи і не переписуючи половину проєкту.



Богдан Левунець
Senior Gameplay Programmer Ubisoft Kyiv

План

01 Вступ

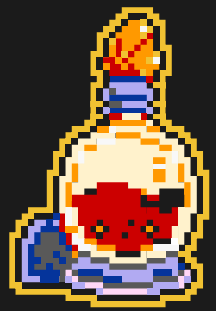
02 Інструменти

03 Процедурний підхід

04 Проблеми

05 ООП

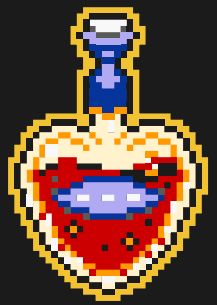
06 Висновки



01

Вступ та Мотивація

Чому розробка ігор - гарна практика ООП ?



Чому розробка ігор — гарна практика ООП ?



Відчутні сутності

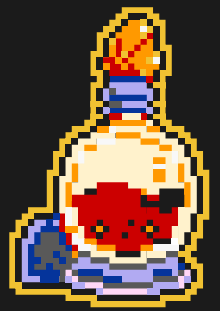
Об'єкти не абстрактні: Гравець, Куля, Ворог, Інтерфейс мають свій стан, координати та життєвий цикл.

Масштабування

ООП розкриває свою силу саме тоді, коли потрібно постійно додавати нову поведінку, не чіпаючи старий код.

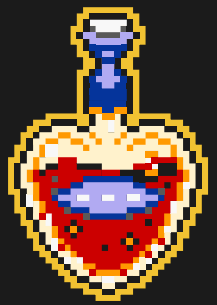
Складна зв'язність

Маленьку гру легко написати в одному файлі. Але коли з'являється 10+ механік, процедурний код стрімко перетворюється на хаос із глобальних змінних та switch.



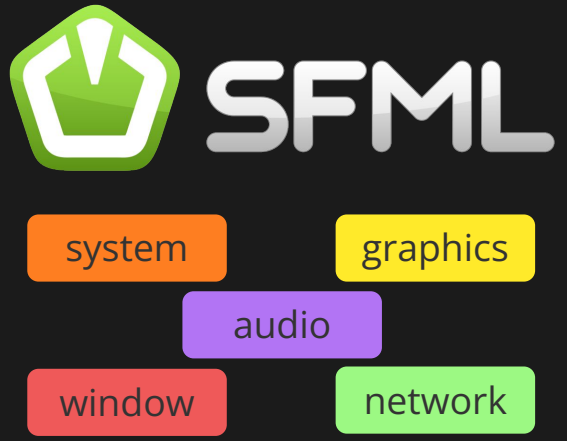
02

Інструментарій



SFML 3.0 - Simple Fast Multimedia Library

ПЕРЕВАГИ

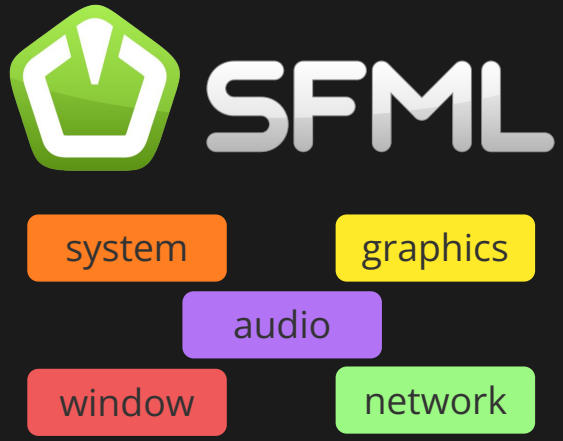




SFML 3.0: Мультимедійний каркас для C++

- **Чистий C++ дизайн API**: SFML написана на сучасному ідіоматичному C++, що привчає до гарного кодстайлу та стандартних патернів проєктування.
- **Мінімальний поріг входження**: Порівняно з DirectX чи OpenGL, вікна, примітивні фігури створюються у декілька рядків без вивчення громіздких пайплайнів відеокарти.
- **Фокус на архітектурі, а не на контенті:** Завдяки вбудованим примітивам можна створити повноцінну гру без сторонніх асетів, графічних редакторів та проблем із шляхами до файлів.
- **Кросплатформність**: Працює на Windows, Linux та macOS.

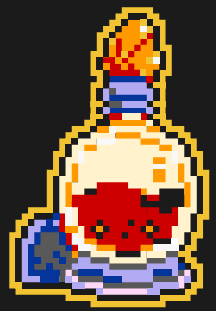
НЕДОЛІКИ





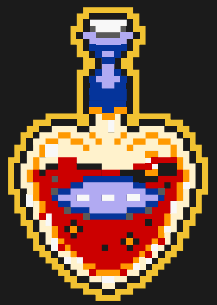
SFML 3.0: Мультимедійний каркас для C++

- **Тільки 2D-графіка:** SFML не підходить для вивчення 3D-математики (матриць трансформації, 3D-шейдерів) без самостійного низькорівневого підключення OpenGL.
- **Бібліотека, а не повноцінний ігровий рушій:** В SFML немає готового фізичного рушія (як Box2D), тайлових карт, системи анімацій чи готових UI-компонентів (кнопок, полів вводу). Усе це доводиться реалізовувати самостійно або підключати сторонні рішення.
- **Межа для масштабних проєктів:** При переході до складних ігор (тисячі одночасних об'єктів) класичний підхід ООП через ієрархії успадкування починає програвати за швидкодією підходам Data-Oriented Design



03

Ігровий цикл



Як оживити код: перехід від консолі до інтерактивного світу

Анатомія ігрового циклу

Game Loop



01

Обробка Інпуту

закриття вікна, натиски миші, клавіатури



02

Оновлення світу

Математика переміщень, таймери перезарядки, обрахунок зіткнень

03

Рендеринг

Очищення попереднього кадру, малювання всіх об'єктів та показ на моніторі.

Ігровий цикл

Game Loop

```cpp
//GAME LOOP
while (window.isOpen())
{
    //Обробка подій ігрового вікна
    while (const auto event = window.pollEvent())
    {
        if (event->is<sf::Event::Closed>())
            window.close();

        if (const sf::Event::KeyPressed* keyPressed = event->getIf<sf::Event::KeyPressed>())
        {
            if (keyPressed->code == sf::Keyboard::Key::Escape)
                window.close();
        }
    }

    //(Input Processing) Обробка інпут девайсів
    sf::Vector2f movement{ 0.f, 0.f };
    if (sf::Keyboard::isKeyPressed(sf::Keyboard::Key::W))
        movement.y -= 1.f;
    if (sf::Keyboard::isKeyPressed(sf::Keyboard::Key::S))
        movement.y += 1.f;
    if (sf::Keyboard::isKeyPressed(sf::Keyboard::Key::A))
        movement.x -= 1.f;
    if (sf::Keyboard::isKeyPressed(sf::Keyboard::Key::D))
        movement.x += 1.f;

    //Game Update (оновлення ігрової логіки)
    player.move(movement * playerSpeed);

    window.clear(sf::Color(25, 25, 30));

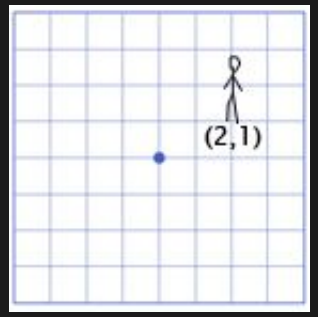
    //Рендер
    window.draw(player);
    window.display();
}
```

2D Вектор

01

Позиція

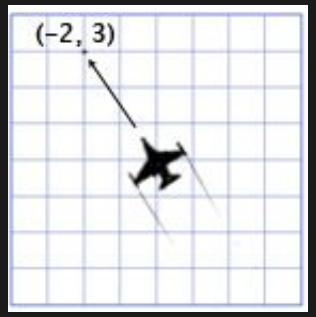
Де саме об'єкт знаходиться на екрані відносно початку координат (0, 0) у пікселях.



02

Швидкість

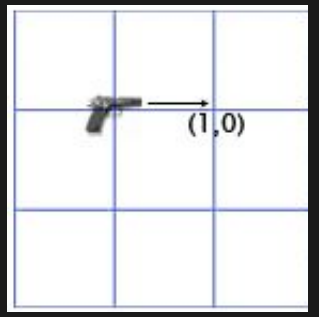
На скільки пікселів і в який бік зміщується тіло за одиницю часу (наприклад, за одну секунду).



03

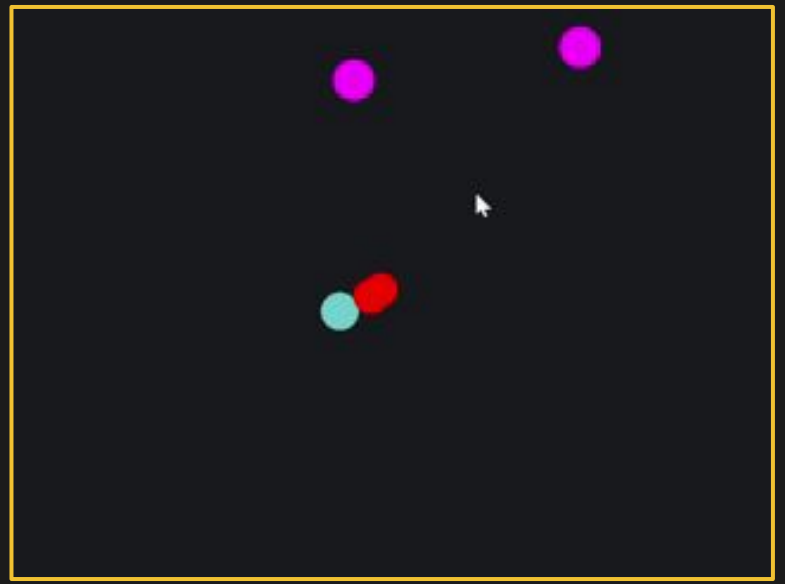
Напрям

Куди “дивиться” чи рухається об'єкт. Чистий вектор фіксованої **одиничної** довжини без прив'язки до швидкості.



04

Процедурний підхід



Структура проєкту

- **struct PlayerData** (відкриті координати, HP)
- **struct EnemyData** (велика група полів)
- **struct BulletData** (налаштування, тип колізії)
- **main.cpp** на 250 рядків коду

Ілюзія простоти

«На старті це виглядає зручно: мінімум файлів, жодного зайвого коду й класів. Але як тільки ми додаємо бодай дві нові механіки — ця простота обертається глухим кутом.»

```cpp
struct PlayerData {
    sf::CircleShape shape;
    float speed = 300.f;
    int health = 100;
    float hitFlashTimer = 0.f;
};
```

```cpp
struct EnemyData {
    sf::CircleShape shape;
    float speed;
    int health;
    EnemyType type;
    float shootCooldown = 0.f;
};
```

```cpp
struct BulletData {
    sf::CircleShape shape;
    sf::Vector2f velocity;
    bool isEnemy = false;
};
```

```cpp
231
232     for (const auto& e
233
234     window.display();
235 }
236
237 return 0;
238 }
```

⚠ Наслідки:

- Роздуті файли
- Спагеті-код у main()
- Складність розширення
- Важка відладка

Процедурний біль: Проблема 1

Роздуті структури даних

Марнування пам'яті

Кожен екземпляр ворога тягне поля, якими ніколи не користується.

Порушення узгодженості (ізоляції)

Будь-хто ззовні може випадково змінити shootCooldown звичайного переслідувача.

```
struct EnemyData {
    sf::CircleShape shape;
    float speed;
    int health;
    EnemyType type;
    float shootCooldown = 0.f;

    // Потім вирішили додати нові типи ворогів
    float shieldEnergy;    // лише типу Танк
    float ammoCount;

    float explosionTimer; // лише Камікадзе
    float explosionRaduis;
};
```

struct EnemyData

Describe with Copilot

Size: 376 bytes

Alignment: 8 bytes

Memory Layout



Процедурний біль: Проблема 2

Всюдисущий switch та дублювання

Дублювання розгалужень

Такий самий switch дублюється у функціях `drawEnemy`, `spawnEnemy`, `collideEnemy` -> додавання/видалення/зміна одного типу вимагає оновлення всіх цих частин коду

Порушення узгодженості (ізоляції)

Зміна одного типу ворога створює ризик зламати код усіх інших типів.

```
void updateEnemies(
    std::vector<EnemyData>& enemies,
    std::vector<BulletData>& bullets,
    const sf::Vector2f& playerPos,
    float dt) {
    for (EnemyData& enemy : enemies) {
        switch (enemy.type) {
        case EnemyType::Chaser: {
            //логіка переслідування
            break;
        }
        case EnemyType::Shooter: {
            //дистанція + постріли
            break;
        //Ще 5 ворогів? -> ще 100 рядків коду в одну функцію!
        }
        }
    }
}
```



Процедурний біль: Проблема 3

Параметричний вибух: пастка передачі контексту

Протягування аргументів

Функції верхнього рівня змушені знати про всі залежності нижнього рівня і «транзитом» тягнути їх крізь весь стек викликів

Паразитна зв'язність (High Coupling)

Типу Chaser кулі не потрібні, але функція змушена вимагати доступ до них. Додавання нової механіки ламає сигнатуру у всіх файлах.

```
void updateEnemies(
    std::vector<EnemyData>& enemies,
    std::vector<BulletData>& bullets,
    const sf::Vector2f& playerPos,
    float dt) {
```



```
void updateEnemies(
    std::vector<EnemyData>& enemies,
    std::vector<BulletData>& bullets,
    const sf::Vector2f& playerPos,
    std::vector<Particle>& fx, //Дим, іскри
    const Map& map, //для обходу перешкод
    SoundManager& sounds, //звукові ефекти
    float dt) {
```

Процедурний біль: Проблема 4

Спагеті код: розмиття меж відповідальності

Змішування рівнів абстракції

Низькорівнева геометрична перевірка перетину раптом змінює стан ворогів, внутрішню анімацію гравця та стан всієї гри.

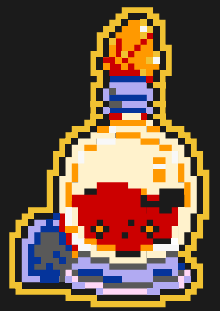
Приховані побічні ефекти (Side Effects)

Функція колізій замість простої фіксації удару також змінює інші стани гри. Наприклад, зміна логіки рестарту гри змушує переписувати та тестувати модуль фізики.

```
if (bullet.isEnemy) {
    //Оновлює внутрішні стани гравця та куль
    if (player.shape.getGlobalBounds()
        .findIntersection(bullet.shape.getGlobalBounds())) {
        player.health -= 20;
        player.hitFlashTimer = 0.15f;
        bullet.shape.setPosition({ -100.f, -100.f });

        // Якщо здоров'я закінчилось — рестарт гри
        // Чомусь скидає стан гри???
        if (player.health <= 0) {
            player.health = 100;
            player.shape.setPosition(
                { WINDOW_WIDTH / 2.f, WINDOW_HEIGHT / 2.f });
            enemies.clear();
            bullets.clear();
            break;
        }
    }
}
```

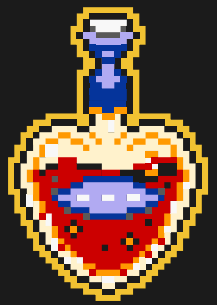




05

ООП

Від Процедурного хаосу до Об’єктного порядку



ООП концепти: Інкапсуляція

Інкапсуляція — спосіб обмеження прямого доступу до компонентів об'єкта. Вона приховує змінні (стан), та внутрішні допоміжні методи класу, гарантуючи взаємодію лише через контрольований інтерфейс.

Приховування даних

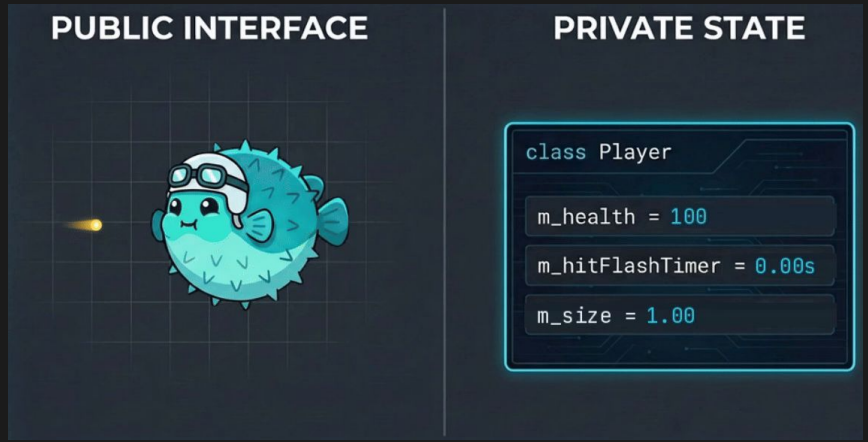
Користувачі не знають деталей реалізації та зберігання. Вони лише передають аргументи й отримують очікуваний результат.

Гнучкість доступу

Дозволяє робити поля лише для читання (get) або лише для запису (set), валідувати вхідні дані та фільтрувати некоректні стани.

Адаптивність коду

Інкапсульований клас легко модифікувати та оптимізувати: зміни внутрішніх структур не ламають решту компонентів програми.



ООП рефакторинг: Крок 1

Перехід до Інкапсуляції

Порівняння: Прямий доступ vs Метод об'єкта

Прямий доступ

Зовнішній код напряму змінює внутрішні змінні гравця. Логіка перезапуску розмазана по всій програмі.

```
//Оновлюємо внутрішні стани гравця
player.health -= 20;
player.hitFlashTimer = 0.15f;
player.shape.setFillColor(sf::Color::Red);

// Якщо здоров'я закінчилось — рестарт гри
if (player.health <= 0) {
    player.health = 100;
    player.shape.setPosition({ 0.0f, 0.0f });
}
```

⚠ Ризики: Некоректні стани, дублювання коду перевірок та відсутність контролю доступу.

Метод об'єкта

Гравець сам керує своїм HP та захищає власні стани.

```
if (player.getBounds().findIntersection(bullet.getBounds())) {
    player.takeDamage(15);

    if (!player.isAlive()) {
        player.reset({ WINDOW_WIDTH / 2.f, WINDOW_HEIGHT / 2.f });
    }
}
```



```
// Інкапсуляція: гравець сам керує своїм HP та візуальною реакцією
void Player::takeDamage(int amount) {
    m_health -= amount;
    if (m_health < 0) m_health = 0;
    m_hitFlashTimer = 0.15f; // Спалах при отриманні шкоди
}

void Player::reset(sf::Vector2f startPos) {
    m_health = 100;
    m_hitFlashTimer = 0.f;
    m_shape.setPosition(startPos);
    updateColor();
}
```

ООП концепти: Композиція

Композиція: зв'язок "**HAS-A**" (Має в собі)

Клас Game World як оркестр об'єктів зі зв'язком HAS-A

Збираємо велику систему з незалежних маленьких деталей.

Ігровий світ не є вікном чи гравцем, він ними володіє, а ті в свою чергу можуть володіти своїми компонентами, формуючи складну структуру з простіших елементів.



ООП рефакторинг: Крок 2

Перехід до Композиції

Клас Game як оркестр 

- **Game** has-a **Window:** володіє вікном рендерингу
- **Game** has-a **Player:** володіє гравцем
- **Game** has-a **Enemies:** володіє списком супротивників

```cpp
class Game {
private:
    // КОМПОЗИЦІЯ (HAS-A):
    // Game володіє цими компонентами
    sf::RenderWindow m_window;
    Player m_player;
    std::vector<Enemy> m_enemies;
    std::vector<Bullet> m_bullets;
```

Було: God-Function main()

```cpp
233            window.display();
234        }
235
236        return 0;
237    }
238
```

Стало: Чистий запуск run методу

```cpp
2
3   int main() {
4       Game game;
5       game.run();
6       return 0;
```

ООП концепти: Наслідування

Наслідування (Inheritance) — здатність класу успадковувати властивості та характеристики іншого класу. Зв'язок **"IS-A"**

Рівень 1

«Enemy» (Базовий клас)

Тільки дані:
+ health
+ position
+ sprite

Спільне для всіх: здоров'я, координати, вигляд.



наслідує / є одним із (IS-A)

Рівень 2

MeleeEnemy (Ближній бій)

Додає:
+ speed
+ damage

Додає швидкість і силу атаки.



RangedEnemy (Дальній бій)

Додає:
+ shootCooldown
+ range

Додає атаку на відстані.



наслідує / є одним із (IS-A)

наслідує / є одним із (IS-A)

Рівень 3

ChaserEnemy (Переслідувач)

Додає:
+біг на гравця
+укус при контакті

Наслідує MeleeEnemy



KamikazeEnemy (Камікадзе)

Додає:
+шалене обертання
+самовибух

Наслідує MeleeEnemy



ShooterEnemy (Стрілець)

Додає:
+утримання дистанції
+стрільба

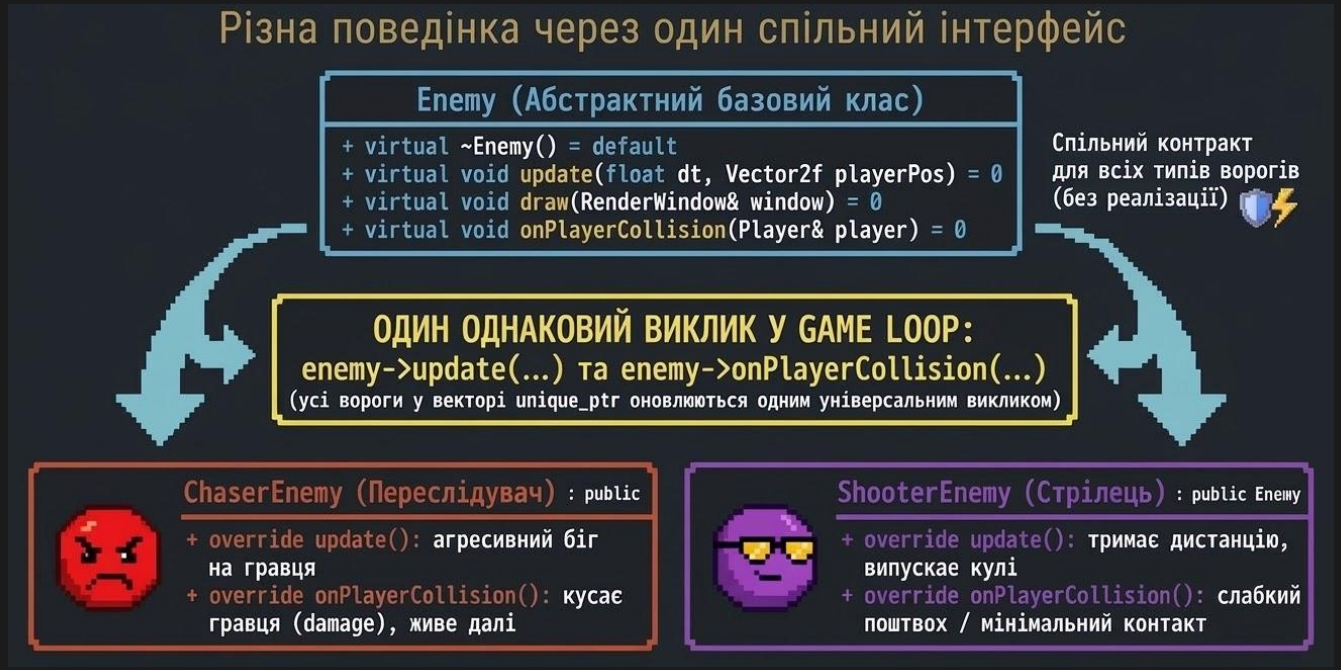
Наслідує RangedEnemy



ООП концепти: Поліморфізм

Поліморфізм означає «багато форм», і він виникає, коли існує багато класів, пов’язаних між собою через наслідування.

Успадкування дозволяє нам отримувати атрибути та методи з іншого класу. Поліморфізм використовує ці методи для виконання різних завдань. Це дає можливість виконувати одну й ту саму дію різними способами.



1. БУЛО (без поліморфізму)

Довгий перебір типів ворогів кожного разу, коли код стосується будь-чого з класу Enemy.

```
if (m_type == EnemyType::Chaser) {
    m_shape.move(dir * m_speed * dt);
} else if (m_type == EnemyType::Shooter) {
    if (dist > 250.f) {
        m_shape.move(dir * m_speed * dt);
    }
    m_shootCooldown -= dt;
    if (m_shootCooldown <= 0.f) {
```



ООП рефакторинг: Крок 3

2. СТАЛО (з поліморфізмом)

- **Поліморфний виклик:** код однаковий **НЕЗАЛЕЖНО** від типу.
- Зміна або додавання ворогів не змінює «код користувача».

Універсальний виклик

Один рядок коду обробляє будь-який тип ворога в єдиному циклі

```cpp
//Game::update()
// ПОЛІМОРФНИЙ ВИКЛИК
for (auto& enemy : m_enemies) {
    enemy->update(dt, m_player.getPosition(), m_bullets);
}
```

```cpp
void ShooterEnemy::update(float dt, sf::Vector2f playerPos, std::vector<Bullet>& outBullets) {
    if (m_hitFlashTimer > 0.f) m_hitFlashTimer -= dt;
    updateColor();

    sf::Vector2f dir = playerPos - m_shape.getPosition();
    float dist = std::sqrt(dir.x * dir.x + dir.y * dir.y);
    if (dist != 0.f) dir /= dist;

    if (dist > 250.f) {
        m_shape.move(dir * m_speed * dt);
    }
```

Специфічна поведінка: ShooterEnemy

```cpp
void ChaserEnemy::update(float dt, sf::Vector2f playerPos, std::vector<Bullet>& /*outBullets*/) {
    if (m_hitFlashTimer > 0.f) m_hitFlashTimer -= dt;
    updateColor();

    sf::Vector2f dir = playerPos - m_shape.getPosition();
    float dist = std::sqrt(dir.x * dir.x + dir.y * dir.y);
    if (dist != 0.f) dir /= dist;

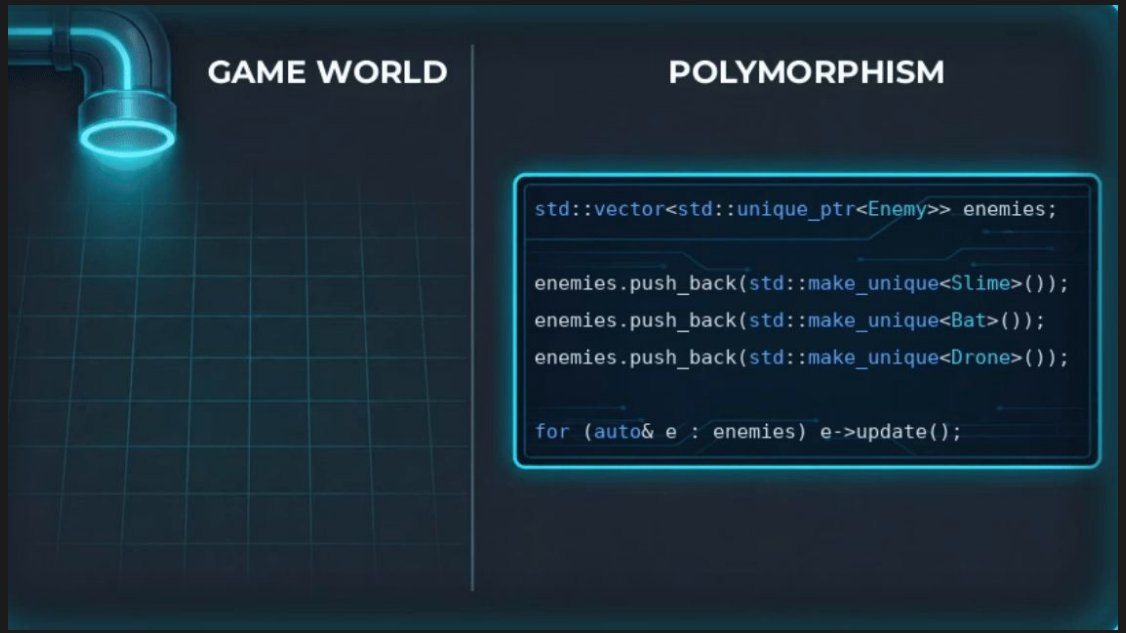
    m_shape.move(dir * m_speed * dt);
}
```

Специфічна поведінка: ChaserEnemy

ООП рефакторинг: Крок 3

Єдине місце, де фігурує конкретний клас - при створенні нових ворогів

```
if (std::rand() % 2 == 0) {
    m_enemies.push_back(std::make_unique<ChaserEnemy>(spawnPos));
} else {
    m_enemies.push_back(std::make_unique<ShooterEnemy>(spawnPos));
}
```



ООП концепти: Додавання нових підкласів

ДОДАВАННЯ НОВИХ ВОРОГІВ

01

Унаслідуватись від базового класу **Enemy**



02

Додати **унікальні дані** для цього класу



03

Реалізувати **віртуальні методи**



04

У потрібному місці створити **об’єкт** нового класу



05

Profit!
(Успіх)



ООП концепти: Поліморфізм

Принцип відкритості/закритості(OCP)

SOLID в дії

«Програмні сутності мають бути відкритими для розширення, але закритими для модифікації».



Стабільність Game Loop

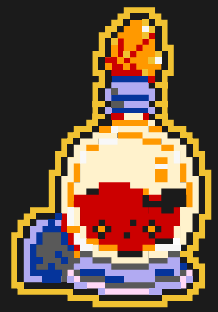
Архітектура захищена від випадкових помилок нових механік.

У класі Game не змінюється ЖОДЕН рядок циклів чи колізій!

Live Coding: Let's add Kamikaze Enemy

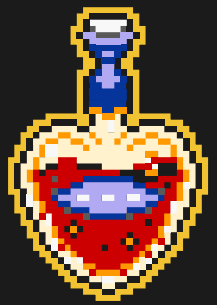


06



Висновки

Переваги та недоліки ООП



Висновки

Еволюція коду: 5 кроків рефакторингу

ЕТАП	АРХІТЕКТУРА	ГОЛОВНА ПРОБЛЕМА	РІШЕННЯ В C++
01. Loop	Базовий SFML Game Loop	Немає динаміки сутностей	Векторна нормалізація, dt
02. Procedural	EnemyData + switch(type)	Роздуті структури, спагеті колізій	Усвідомлення меж процедурного стилю
03. Encapsulation	Класи з private-полями	main() перевантажений кодом	Методи takeDamage(), захист стану
04. Composition	Клас Game (зв'язок HAS-A)	Важко додавати нових ворогів	Ізоляція циклу: update / render
05. Polymorphism	Інтерфейс Enemy + unique_ptr	Умовно відсутні	Поліморфні списки, чистий інтерфейс

Висновки

Прагматичний інженерний вибір: Чи завжди потрібне ООП?



Де ООП сяє найкраще

- **Системи з високою варіативністю типів поведінки** (ігрові сутності, вороги, зброя).
- **Користувацькі інтерфейси** (GUI / UI віджети).
- **Парсери форматів** та бізнес-логіка додатків.



Де перемагає DOD / ECS

- **Математичні розрахунки** та шейдери.
- **Симуляція** сотень тисяч однорідних частинок.
- **Data-Oriented Design та ECS** (Entity Component System) для кеш-локальності процесора.

THANKS!

Do you have any questions?

Code repository

https://github.com/bohdanpc/OOP_SFML_Lecture

